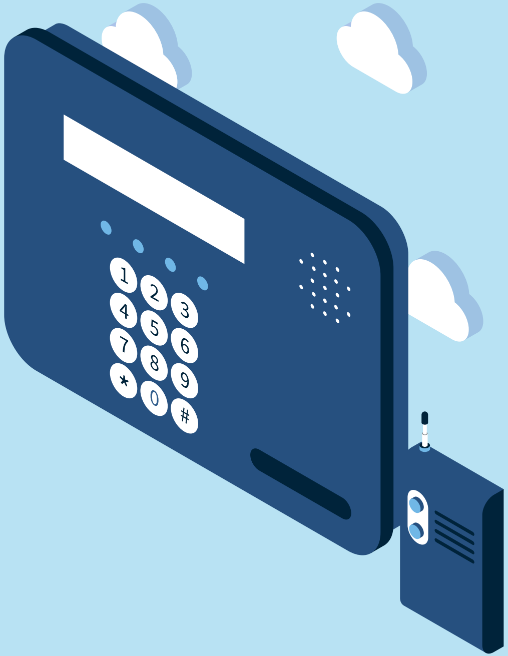
# COMBATTING NEIGHBORHOOD THEFTS

## – A DATA ENGINEERING APPROACH

# CONTENTS

# OVERVIEW

Neighborhood thefts pose a significant challenge to the safety and well-being of our community. In this project, we leveraged the power of data engineering via GCP to uncover insights and develop strategies to address this pressing issue



# INTRODUCTION

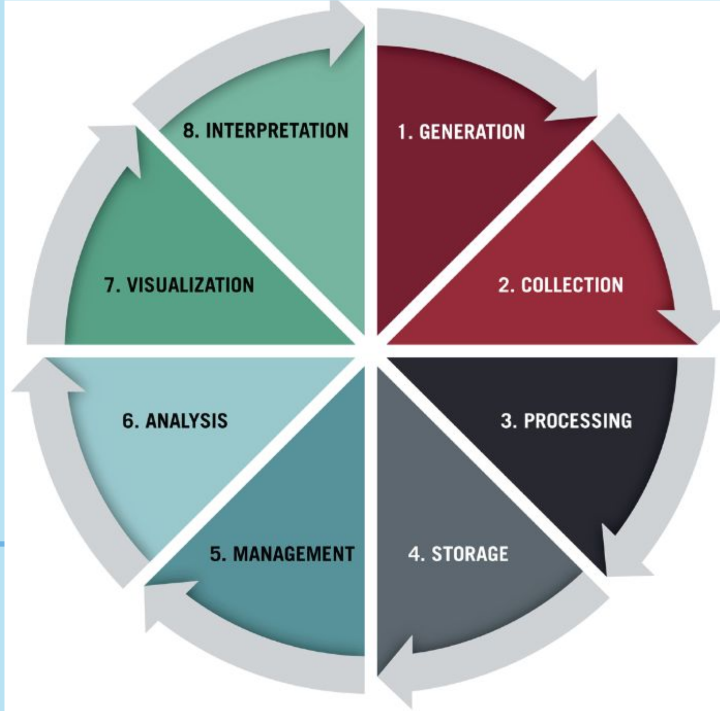
**The Problem:**

With rising temperatures during the summer, so do the  property thefts in the neighborhood. With longer days and vacations, homes become vulnerable, and criminals seize the opportunity.

**Importance:**

By understanding the underlying patterns and factors contributing to neighborhood thefts, we can empower local authorities and community leaders to implement targeted solutions that enhance safety and improve the quality of life for Denton residents.

# IMPLEMENTATION THROUGH DATA LIFECYCLE




**Identified Sources in the form of static datasets**

**Open Refine**

**Google Cloud Storage**

**Google Cloud Platform**

Big Query, Hive & Spark SQL

# DATA GENERATION & COLLECTION

We identified several key sources for generating data related to neighborhood thefts. Some of them include:

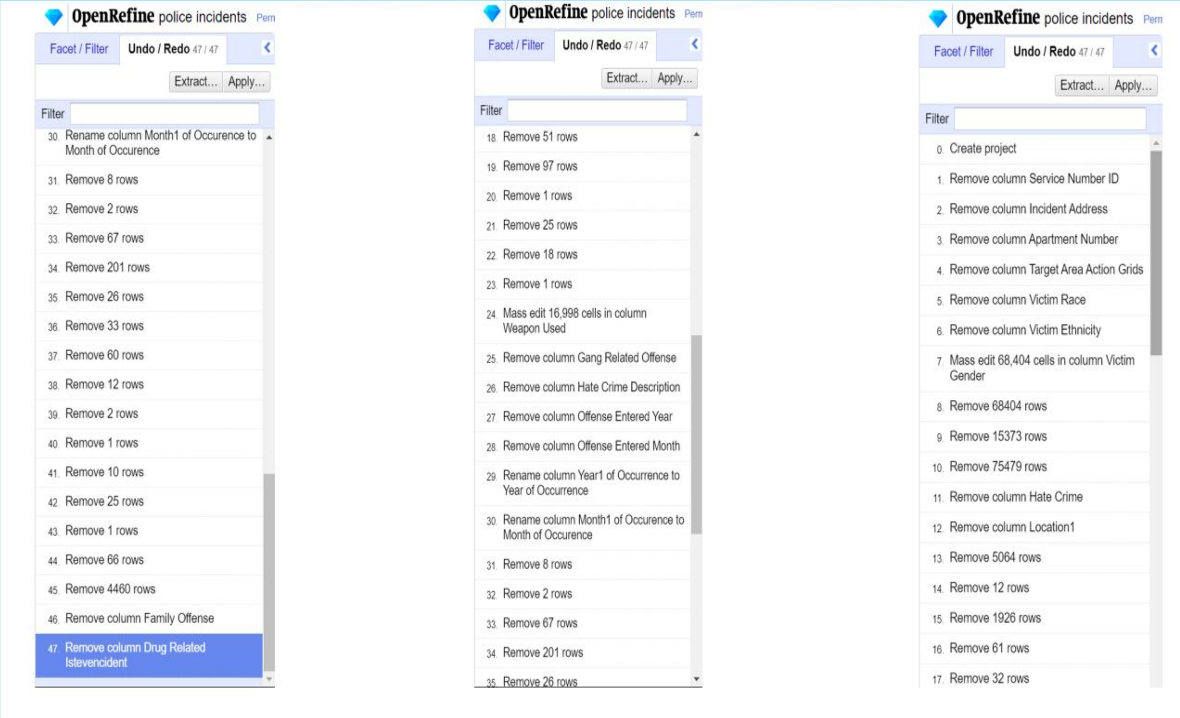
- Police Department Records
- Community Reports
- Sensor data

By utilizing these diverse data sources and collection methods, we were able to assemble 3 static datasets that provided valuable insights into the patterns and contributing factors behind neighborhood thefts in Denton.

- https://communitycrimemap.com/
- https://data.syr.gov/datasets/5b53f40abbc54c559ca991c96816cc17_0/about
- https://www.dallasopendata.com/Public-Safety/Police-Incidents/qv6i-rri7/about_data

Data generation and collection. (n.d.). Shutter Stock.

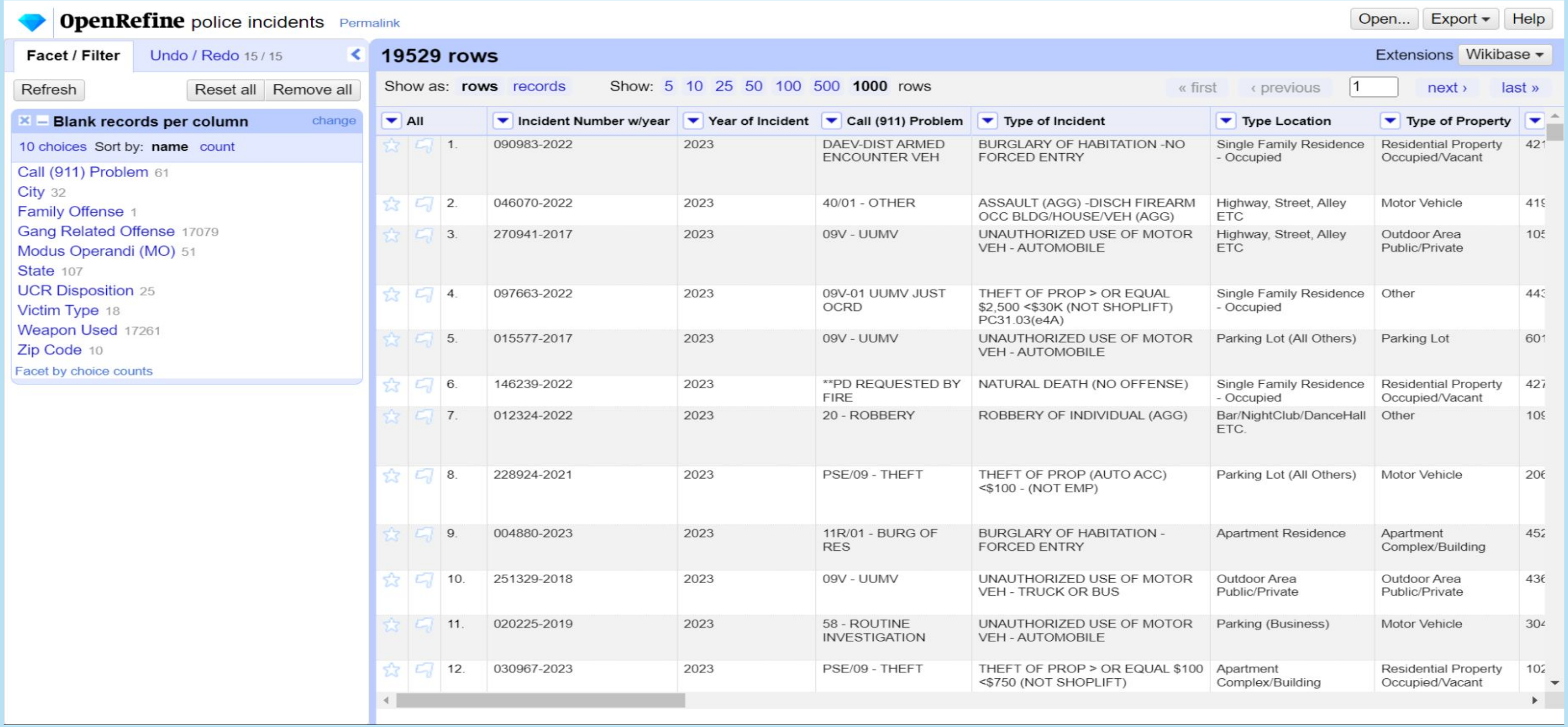
# DATA PROCESSING USING OPEN REFINE




We employed **OpenRefine** to ensure the quality and integrity of our data, we Data Cleaning and Preparation Techniques we used:

OpenRefine helped us remove duplicates, standardize data formats, and handle missing values, by ensuring a clean and consistent dataset for further analysis.

We also utilized OpenRefine's advanced features to detect and resolve inconsistencies in the data, enhancing the overall data quality.



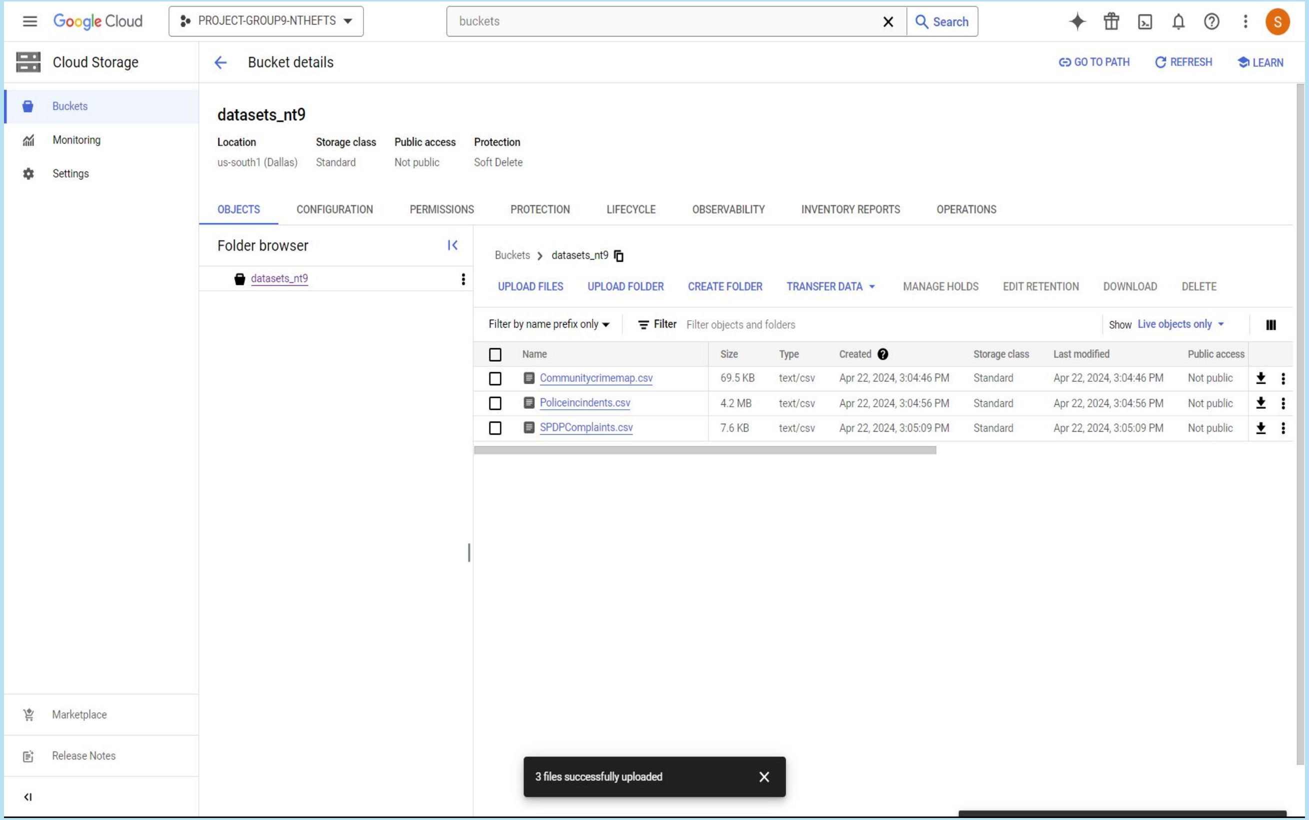

Screenshot – OpenRefine interface Selected blank facets as part of data cleaning process.

# DATA STORAGE

Upon cleaning and processing the data, we utilized Google Cloud Storage buckets to store the datasets.

Cloud Storage provided us with a centralized and easily accessible repository for the processed data, facilitating seamless data sharing and integration across various GCP services throughout our data engineering pipeline.

Screenshot by Suprathika Vangari




Screenshot - Datasets uploaded into Google Cloud Storage Buckets.

Processed data was stored in Google Storage Buckets

Screenshot - Data Storage Bucket management in Google Cloud Storage

# DATA MANAGEMENT

As part of a crucial step in our data engineering pipeline, we leveraged Google Cloud's Dataproc service to set up a robust Hadoop ecosystem for managing and processing the neighborhood theft data.

Dataproc allowed us to set-up a Hadoop cluster tailored to our project's requirements.

We configured the cluster with the following specifications:

- Cluster Name: "dataproc-Hadoop-spark-cluster"
- Region and Zone: us-central1
- Manager Node: e2-standard-4 machine type with 128 GB disk size
- Worker Nodes: 2 nodes with 128 GB disk size
- By utilizing Hadoop environment (HDFS) managed by Dataproc, we were able to focus on the data engineering tasks.

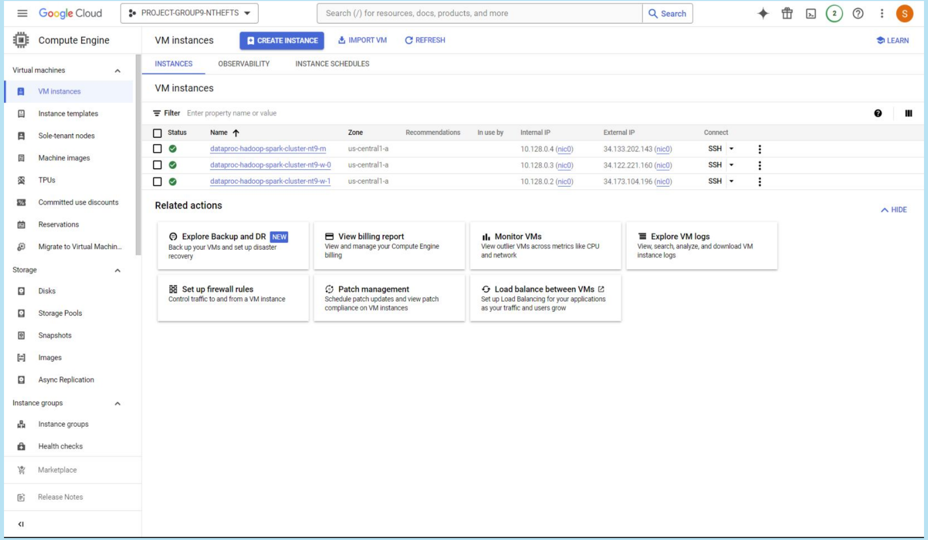
# DATA MANAGEMENT

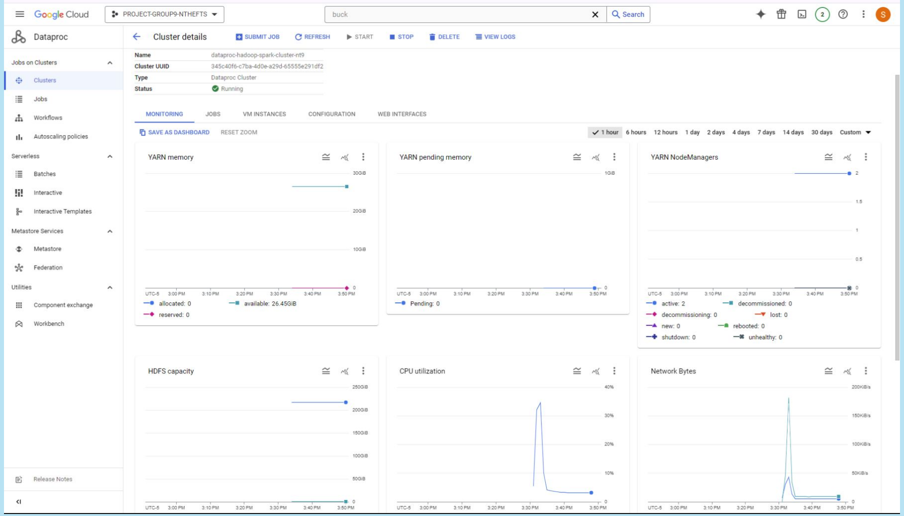
The Dataproc cluster served as the foundation for various stages of our data engineering workflow:

- Data Processing: We used the Hadoop Distributed File System (HDFS) within the Dataproc cluster to store the refined datasets, ensuring scalable and reliable data storage.
- Data Analysis: Leveraging Apache Spark and Apache Hive, both of which are pre-installed on the Dataproc cluster, we were able to perform advanced analytics and extract valuable insights from the neighborhood theft data.
- Data Management: The Dataproc cluster provided us with a centralized and easily accessible platform for managing the data and running queries to support the project's objectives.

By deploying the Dataproc cluster within the Google Cloud ecosystem, we were able to integrate it with other GCP services, such as Cloud Storage and BigQuery.

# Integration with GCP:

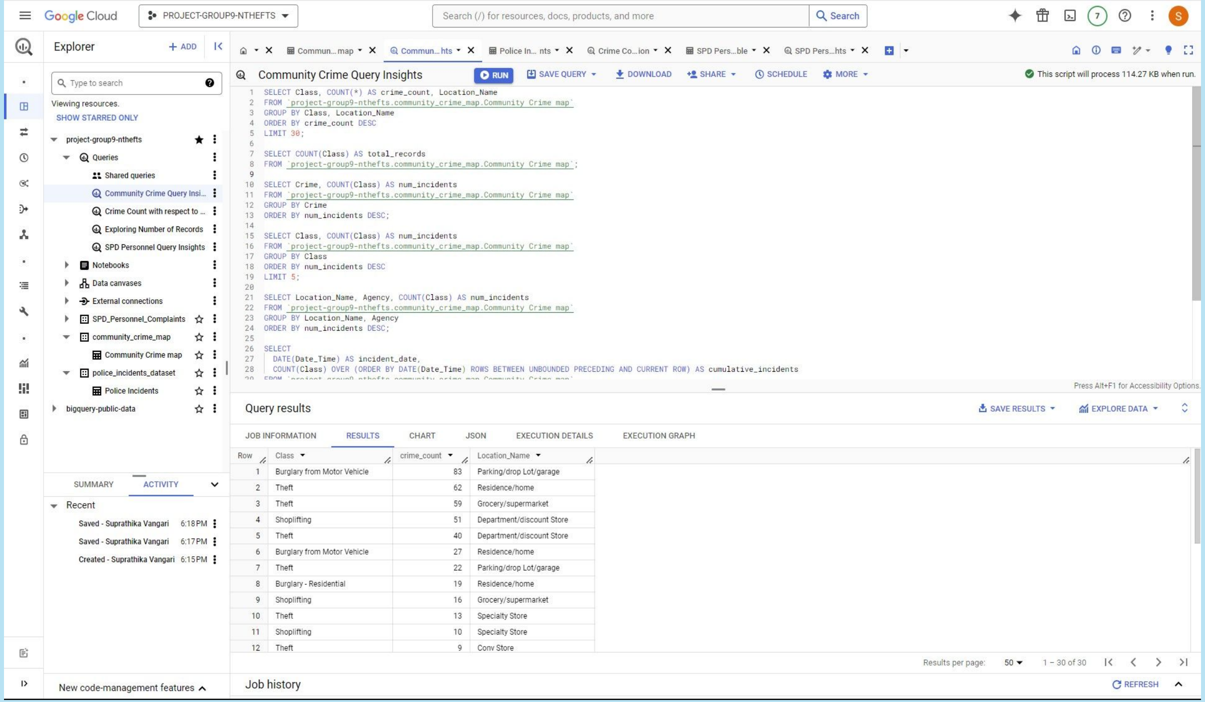


# DATA MANAGEMENT WITH BIGQUERY

To manage and query the processed data effectively, we utilized BigQuery -GCP's fully-managed enterprise data warehouse solution.

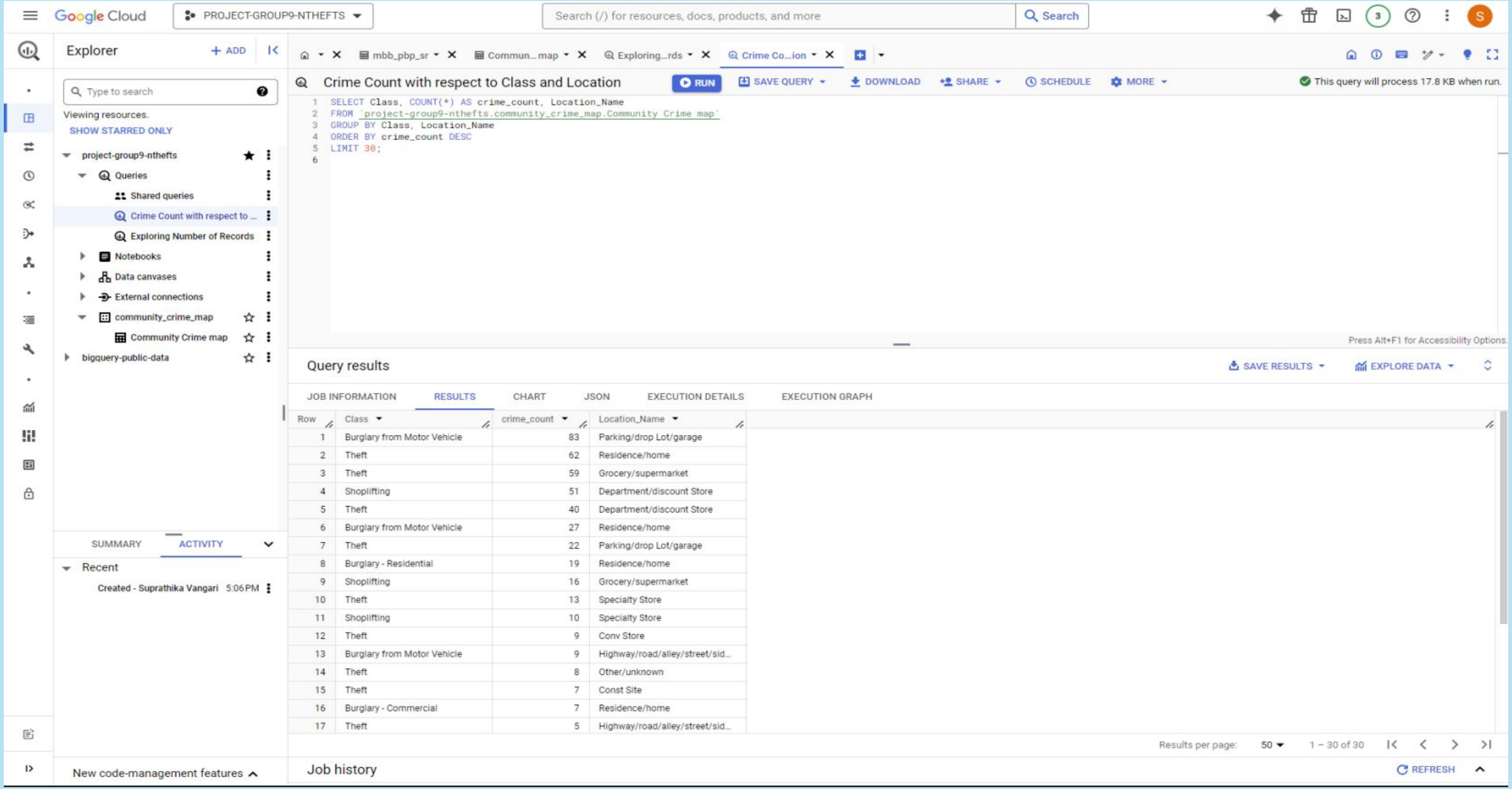
By creating an external table in BigQuery, we were able to directly access the data stored in our Cloud Storage bucket, streamlining the data management process and enabling us to run sophisticated SQL queries.




Screenshot of querying using Bigquery and results

Screenshot by Suprathika Vangari

Querying on 3 datasets using BigQuery.

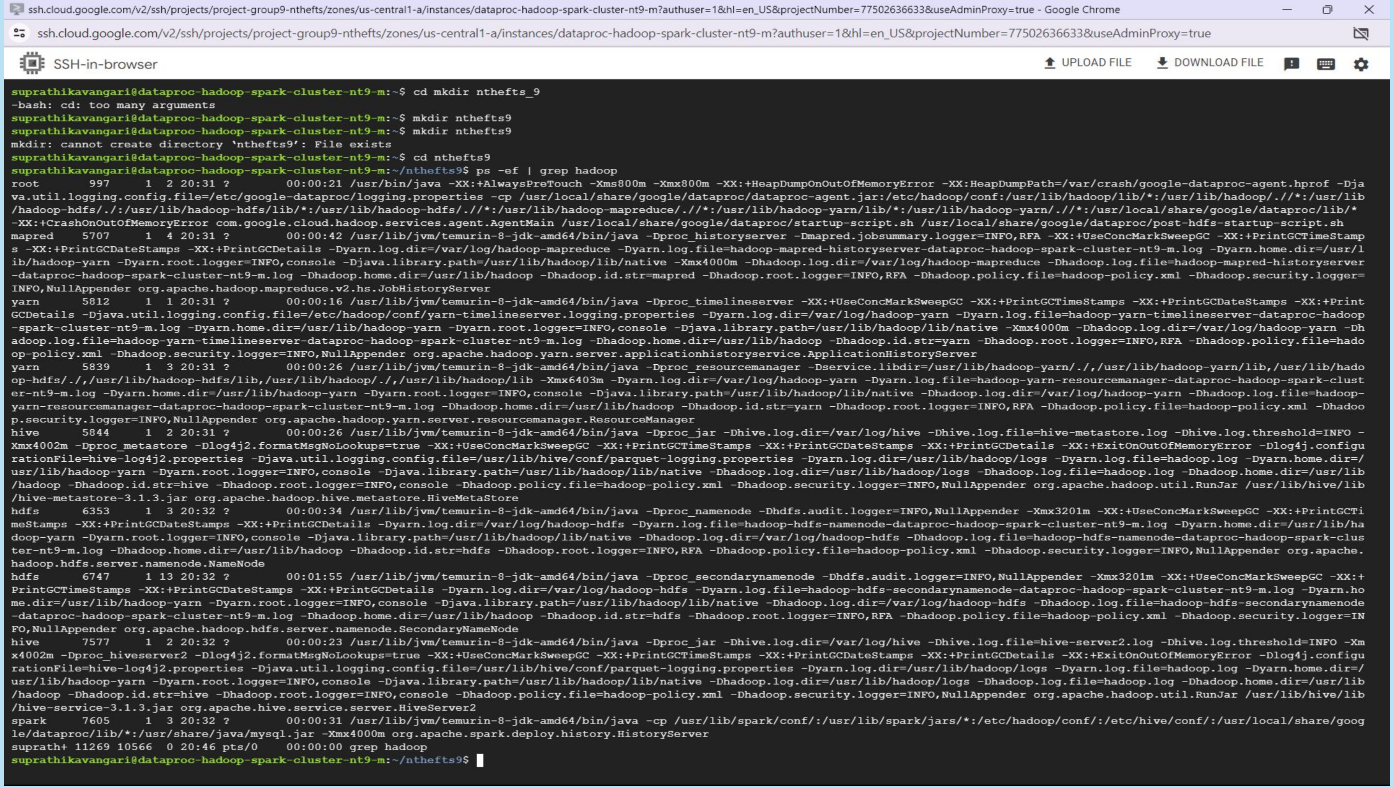



Screenshot – Resultant Crime Class according to crime count and location

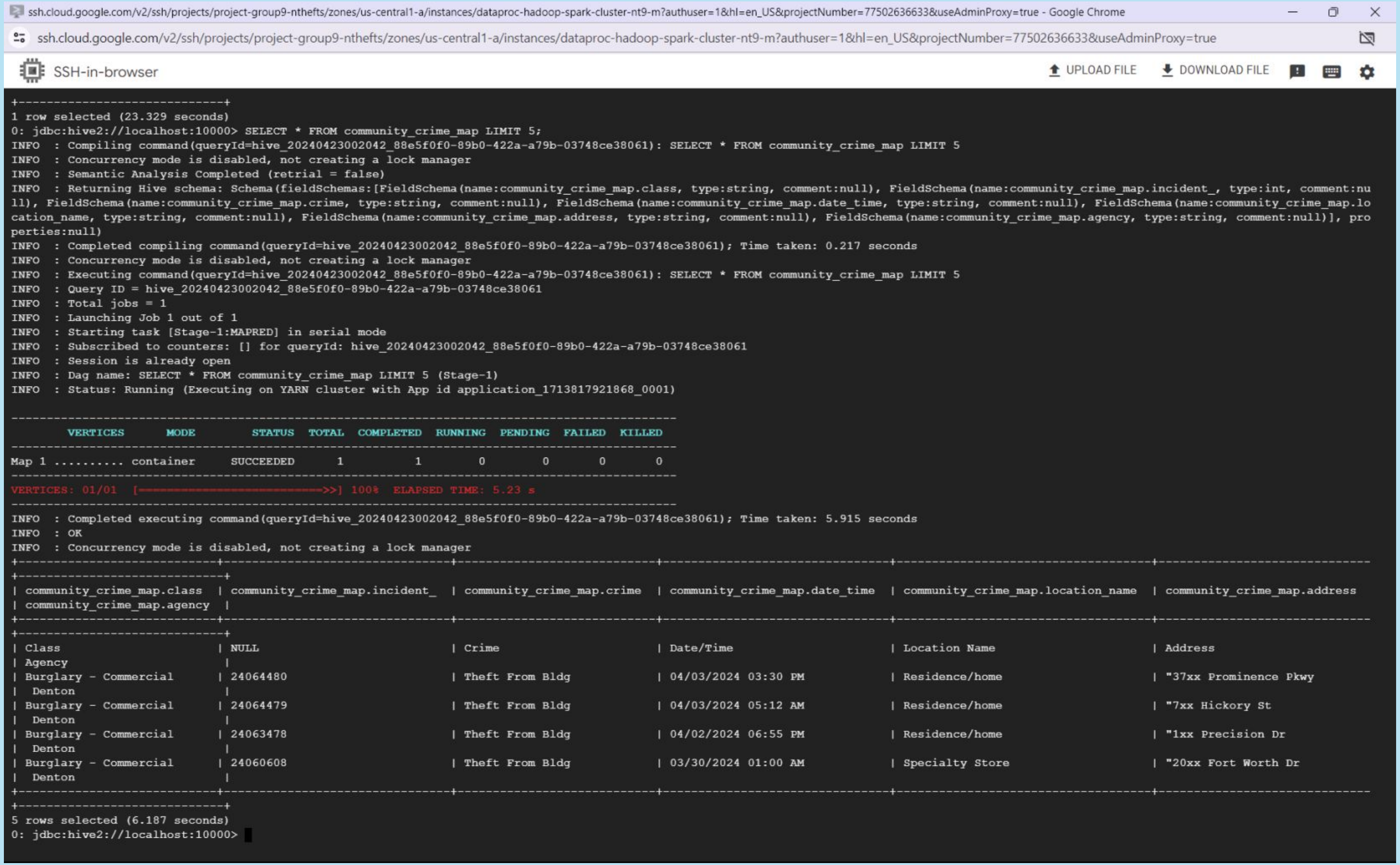
By Suprathika Vangari

# ADVANCED ANALYSIS WITH HIVE & SPARK

To perform advanced analysis on the neighborhood theft data, we utilized Apache Hive, a data warehouse software that provides a SQL-like interface for querying and managing structured data



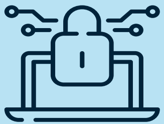

```
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~$ cd mkdir nthefts_9
-bash: cd: too many arguments
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~$ mkdir nthefts9
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~$ mkdir nthefts9
mkdir: cannot create directory ‘nthefts9’: File exists
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~$ cd nthefts9
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~/nthefts9$ ps -ef | grep hadoop
root       997     1  2 20:31 ?        00:00:21 /usr/bin/java -XX:+AlwaysPreTouch -Xms800m -Xmx800m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/crash/google-dataproc-agent.hprof -Dja
va.util.logging.config.file=/etc/google-dataproc/logging.properties -cp /usr/local/share/google/dataproc/dataproc-agent.jar:/etc/hadoop/conf:/usr/lib/hadoop/lib/*:/usr/lib/hadoop/.//*:/usr/lib
/hadoop-hdfs/./:/usr/lib/hadoop-hdfs/lib/*:/usr/lib/hadoop-hdfs/.//*:/usr/lib/hadoop-mapreduce/.//*:/usr/lib/hadoop-yarn/lib/*:/usr/lib/hadoop-yarn/.//*:/usr/local/share/google/dataproc/lib/*
-XX:+CrashOnOutOfMemoryError com.google.cloud.hadoop.services.agent.AgentMain /usr/local/share/google/dataproc/startup-script.sh /usr/local/share/google/dataproc/post-hdfs-startup-script.sh
mapred    5707     1  4 20:31 ?        00:00:42 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_historyserver -Dmapred.jobsummary.logger=INFO,RFA -XX:+UseConcMarkSweepGC -XX:+PrintGCTimeStamp
s -XX:+PrintGCDateStamps -XX:+PrintGCDetails -Dyarn.log.dir=/var/log/hadoop-mapreduce -Dyarn.log.file=hadoop-mapred-historyserver-dataproc-hadoop-spark-cluster-nt9-m.log -Dyarn.home.dir=/usr/l
ib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Xmx4000m -Dhadoop.log.dir=/var/log/hadoop-mapreduce -Dhadoop.log.file=hadoop-mapred-historyserver
-dataproc-hadoop-spark-cluster-nt9-m.log -Dhadoop.home.dir=/usr/lib/hadoop -Dhadoop.id.str=mapred -Dhadoop.root.logger=INFO,RFA -Dhadoop.policy.file=hadoop-policy.xml -Dhadoop.security.logger=
INFO,NullAppender org.apache.hadoop.mapreduce.v2.hs.JobHistoryServer
yarn      5812     1  1 20:31 ?        00:00:16 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_timelineserver -XX:+UseConcMarkSweepGC -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps -XX:+Print
GCDetails -Djava.util.logging.config.file=/etc/hadoop/conf/yarn-timelineserver.logging.properties -Dyarn.log.dir=/var/log/hadoop-yarn -Dyarn.log.file=hadoop-yarn-timelineserver-dataproc-hadoop
-spark-cluster-nt9-m.log -Dyarn.home.dir=/usr/lib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Xmx4000m -Dhadoop.log.dir=/var/log/hadoop-yarn -Dh
adoop.log.file=hadoop-yarn-timelineserver-dataproc-hadoop-spark-cluster-nt9-m.log -Dhadoop.home.dir=/usr/lib/hadoop -Dhadoop.id.str=yarn -Dhadoop.root.logger=INFO,RFA -Dhadoop.policy.file=hado
op-policy.xml -Dhadoop.security.logger=INFO,NullAppender org.apache.hadoop.yarn.server.applicationhistoryservice.ApplicationHistoryServer
yarn      5839     1  3 20:31 ?        00:00:26 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_resourcemanager -Dservice.libdir=/usr/lib/hadoop-yarn/./,/usr/lib/hadoop-yarn/lib,/usr/lib/hado
op-hdfs/./,/usr/lib/hadoop-hdfs/lib,/usr/lib/hadoop/./,/usr/lib/hadoop/lib -Xmx6403m -Dyarn.log.dir=/var/log/hadoop-yarn -Dyarn.log.file=hadoop-yarn-resourcemanager-dataproc-hadoop-spark-clust
er-nt9-m.log -Dyarn.home.dir=/usr/lib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Dhadoop.log.dir=/var/log/hadoop-yarn -Dhadoop.log.file=hadoop-
yarn-resourcemanager-dataproc-hadoop-spark-cluster-nt9-m.log -Dhadoop.home.dir=/usr/lib/hadoop -Dhadoop.id.str=yarn -Dhadoop.root.logger=INFO,RFA -Dhadoop.policy.file=hadoop-policy.xml -Dhadoo
p.security.logger=INFO,NullAppender org.apache.hadoop.yarn.server.resourcemanager.ResourceManager
hive      5844     1  2 20:31 ?        00:00:26 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_jar -Dhive.log.dir=/var/log/hive -Dhive.log.file=hive-metastore.log -Dhive.log.threshold=INFO -
Xmx4002m -Dproc_metastore -Dlog4j2.formatMsgNoLookups=true -XX:+UseConcMarkSweepGC -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps -XX:+PrintGCDetails -XX:+ExitOnOutOfMemoryError -Dlog4j.configu
rationFile=hive-log4j2.properties -Djava.util.logging.config.file=/usr/lib/hive/conf/parquet-logging.properties -Dyarn.log.dir=/usr/lib/hadoop/logs -Dyarn.log.file=hadoop.log -Dyarn.home.dir=/
usr/lib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Dhadoop.log.dir=/usr/lib/hadoop/logs -Dhadoop.log.file=hadoop.log -Dhadoop.home.dir=/usr/lib
/hadoop -Dhadoop.id.str=hive -Dhadoop.root.logger=INFO,console -Dhadoop.policy.file=hadoop-policy.xml -Dhadoop.security.logger=INFO,NullAppender org.apache.hadoop.util.RunJar /usr/lib/hive/lib
/hive-metastore-3.1.3.jar org.apache.hadoop.hive.metastore.HiveMetaStore
hdfs      6353     1  3 20:32 ?        00:00:34 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_namenode -Dhdfs.audit.logger=INFO,NullAppender -Xmx3201m -XX:+UseConcMarkSweepGC -XX:+PrintGCTi
meStamps -XX:+PrintGCDateStamps -XX:+PrintGCDetails -Dyarn.log.dir=/var/log/hadoop-hdfs -Dyarn.log.file=hadoop-hdfs-namenode-dataproc-hadoop-spark-cluster-nt9-m.log -Dyarn.home.dir=/usr/lib/ha
doop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Dhadoop.log.dir=/var/log/hadoop-hdfs -Dhadoop.log.file=hadoop-hdfs-namenode-dataproc-hadoop-spark-clus
ter-nt9-m.log -Dhadoop.home.dir=/usr/lib/hadoop -Dhadoop.id.str=hdfs -Dhadoop.root.logger=INFO,RFA -Dhadoop.policy.file=hadoop-policy.xml -Dhadoop.security.logger=INFO,NullAppender org.apache.
hadoop.hdfs.server.namenode.NameNode
hdfs      6747     1 13 20:32 ?        00:01:55 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_secondarynamenode -Dhdfs.audit.logger=INFO,NullAppender -Xmx3201m -XX:+UseConcMarkSweepGC -XX:+
PrintGCTimeStamps -XX:+PrintGCDateStamps -XX:+PrintGCDetails -Dyarn.log.dir=/var/log/hadoop-hdfs -Dyarn.log.file=hadoop-hdfs-secondarynamenode-dataproc-hadoop-spark-cluster-nt9-m.log -Dyarn.ho
me.dir=/usr/lib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Dhadoop.log.dir=/var/log/hadoop-hdfs -Dhadoop.log.file=hadoop-hdfs-secondarynamenode
-dataproc-hadoop-spark-cluster-nt9-m.log -Dhadoop.home.dir=/usr/lib/hadoop -Dhadoop.id.str=hdfs -Dhadoop.root.logger=INFO,RFA -Dhadoop.policy.file=hadoop-policy.xml -Dhadoop.security.logger=IN
FO,NullAppender org.apache.hadoop.hdfs.server.namenode.SecondaryNameNode
hive      7577     1  2 20:32 ?        00:00:23 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -Dproc_jar -Dhive.log.dir=/var/log/hive -Dhive.log.file=hive-server2.log -Dhive.log.threshold=INFO -Xm
x4002m -Dproc_hiveserver2 -Dlog4j2.formatMsgNoLookups=true -XX:+UseConcMarkSweepGC -XX:+PrintGCTimeStamps -XX:+PrintGCDateStamps -XX:+PrintGCDetails -XX:+ExitOnOutOfMemoryError -Dlog4j.configu
rationFile=hive-log4j2.properties -Djava.util.logging.config.file=/usr/lib/hive/conf/parquet-logging.properties -Dyarn.log.dir=/usr/lib/hadoop/logs -Dyarn.log.file=hadoop.log -Dyarn.home.dir=/
usr/lib/hadoop-yarn -Dyarn.root.logger=INFO,console -Djava.library.path=/usr/lib/hadoop/lib/native -Dhadoop.log.dir=/usr/lib/hadoop/logs -Dhadoop.log.file=hadoop.log -Dhadoop.home.dir=/usr/lib
/hadoop -Dhadoop.id.str=hive -Dhadoop.root.logger=INFO,console -Dhadoop.policy.file=hadoop-policy.xml -Dhadoop.security.logger=INFO,NullAppender org.apache.hadoop.util.RunJar /usr/lib/hive/lib
/hive-service-3.1.3.jar org.apache.hive.service.server.HiveServer2
spark     7605     1  3 20:32 ?        00:00:31 /usr/lib/jvm/temurin-8-jdk-amd64/bin/java -cp /usr/lib/spark/conf/:/usr/lib/spark/jars/*:/etc/hadoop/conf/:/etc/hive/conf/:/usr/local/share/goog
le/dataproc/lib/*:/usr/share/java/mysql.jar -Xmx4000m org.apache.spark.deploy.history.HistoryServer
suprath+ 11269 10566  0 20:46 pts/0    00:00:00 grep hadoop
suprathikavangari@dataproc-hadoop-spark-cluster-nt9-m:~/nthefts9$
```

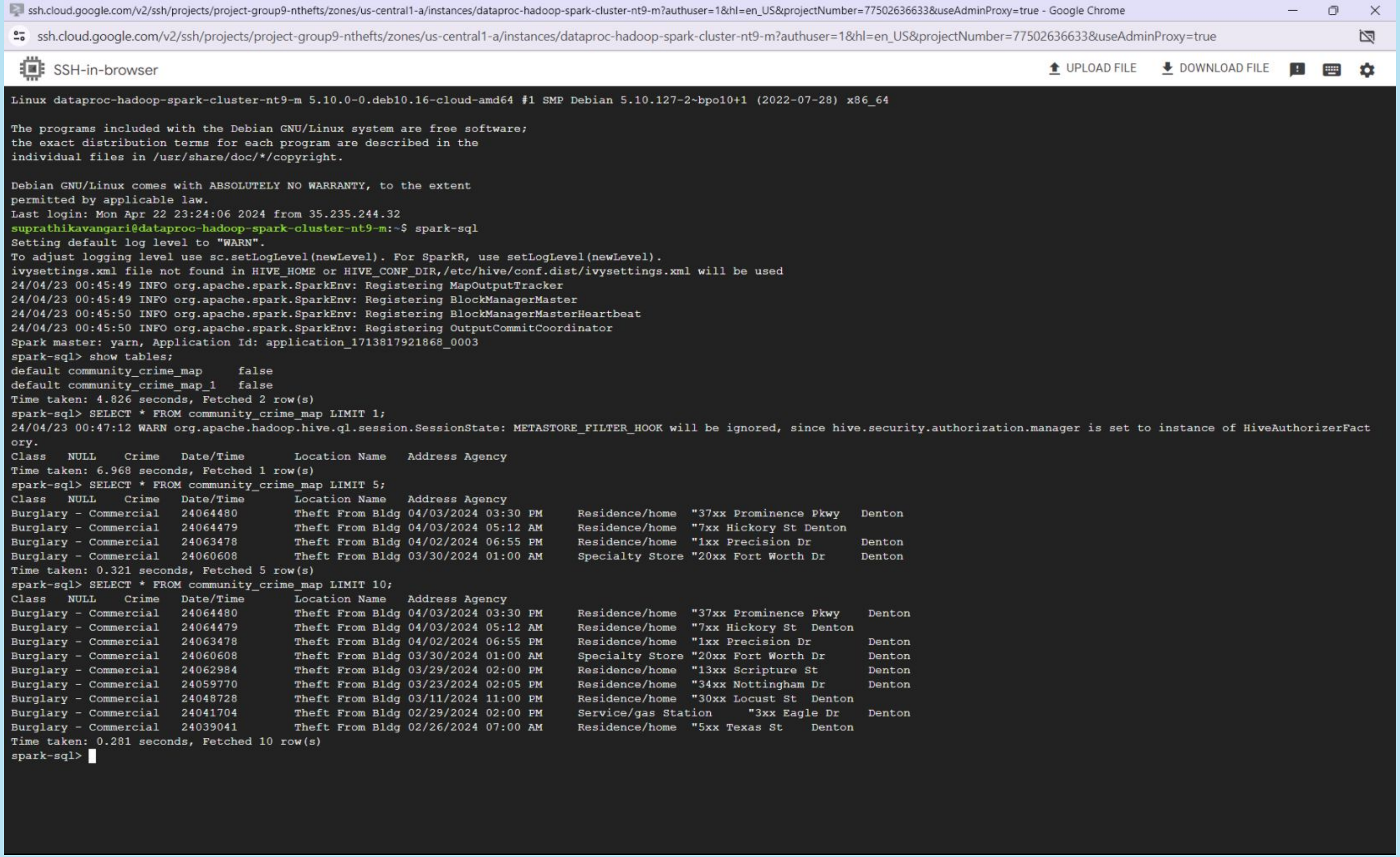



Hive allowed us to define a schema for the data, ensuring a consistent and organized data model for our analysis

Screenshot - Sample HiveQL code snippet for schema definition

In addition to Hive, we leveraged the power of Apache Spark SQL, to develop custom algorithms and queries tailored to our specific analysis needs.






Snippet of Spark Code and its output

# KEY FINDINGS & RECOMMENDATIONS

As data engineers, our role was to establish a robust and efficient data pipeline to support the analysis of neighborhood thefts in Denton.

Through our data engineering efforts, we have laid the foundation for data analysts and scientists to uncover valuable insights and develop targeted solutions.

The data engineering processes we implemented, such as data collection, cleaning, and storage, have provided a high-quality and well-structured dataset for further analysis. Building upon the insights generated from the data pipeline, we have the following recommendations for data analysts and scientists to further explore and implement:

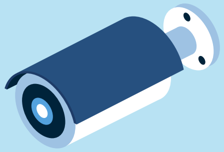
Leverage the community reporting data to establish a neighborhood watch program, empowering residents to actively participate in crime prevention and reporting.

# CONCLUSION

As data engineers, our role has been to establish a comprehensive and scalable data pipeline to support the analysis of neighborhood thefts in Denton. By employing GCP and data engineering best practices, we have created a foundation for data analysts and scientists to uncover valuable insights and develop data-driven solutions.

Moving forward, we encourage the data analysts and scientists to fully utilize the data pipeline we have established.

Through the integration of various data sources, including police records, community reports, and the data processing, storage, and management techniques we implemented ensure the reliability, accessibility, and scalability of the data, empowering the data science team to focus on advanced analysis and insightful recommendations.



# THANK YOU!